

Conditional DETR

The conditional spatial query

How a Detection Transformer learns an order of magnitude faster once we separate the “**what?**” from the “**where?**” inside the decoder’s attention — told readably, but with the technical detail you need.

1. What does DETR do?

DETR (*DE*tecti*on TR*ansformer, Carion et al., 2020) rethought object detection. Unlike classical detectors, it has no pre-defined anchors and no post-hoc *non-maximum suppression*. Instead it frames the task as **set prediction**: the model directly returns a fixed-size set of objects in one shot.

The pipeline has four stages. A **backbone** (e.g. ResNet or DINOv2) turns the image into a feature map; we add a 2D sine **positional encoding** so the Transformer knows where each feature sits. The **encoder** uses self-attention to make every location aware of every other location. The **decoder** receives N learnable “object queries”; each query uses cross-attention to locate its own object in the encoder output. Finally, two small feed-forward heads (FFNs) turn every query into a **class** and a **bounding box**. During training, a bipartite assignment (the *Hungarian matching*) pairs predictions with ground-truth objects.

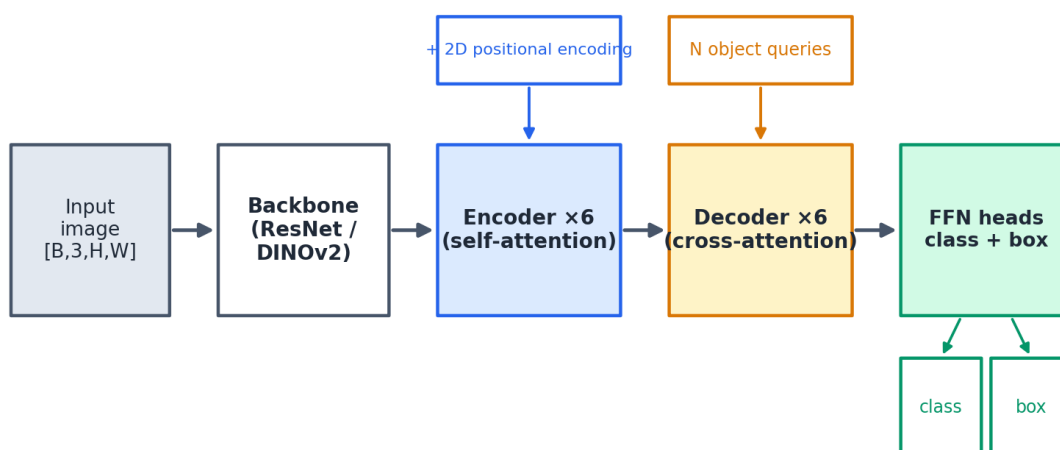


Figure 1 — The DETR pipeline: backbone → encoder → decoder → prediction heads.

The idea in a nutshell

DETR’s elegance comes at the cost of **slow training**: the original model needs roughly 500 epochs. Conditional DETR targets exactly this weakness — with a single, well-aimed change to the decoder’s attention.

2. Why is the original DETR slow?

The slowness lives in the decoder’s **cross-attention**. Here each query tries to find where the object’s key parts are on the image — typically the box edges. The problem is that the query’s spatial “compass” is a single, learned vector that is **shared across all images** (the query positional embedding). On its own it cannot say where the object is in a particular image; for that, the query must rely on its own **content** embedding.

But early in training the content embedding is essentially “empty” — it carries no reliable location information. As a result, the attention stays spread out over the whole image (Figure 2, left) for a long time, and it takes many epochs before the queries even learn where to look. The model effectively learns spatial localization **from scratch**, which is extremely slow.

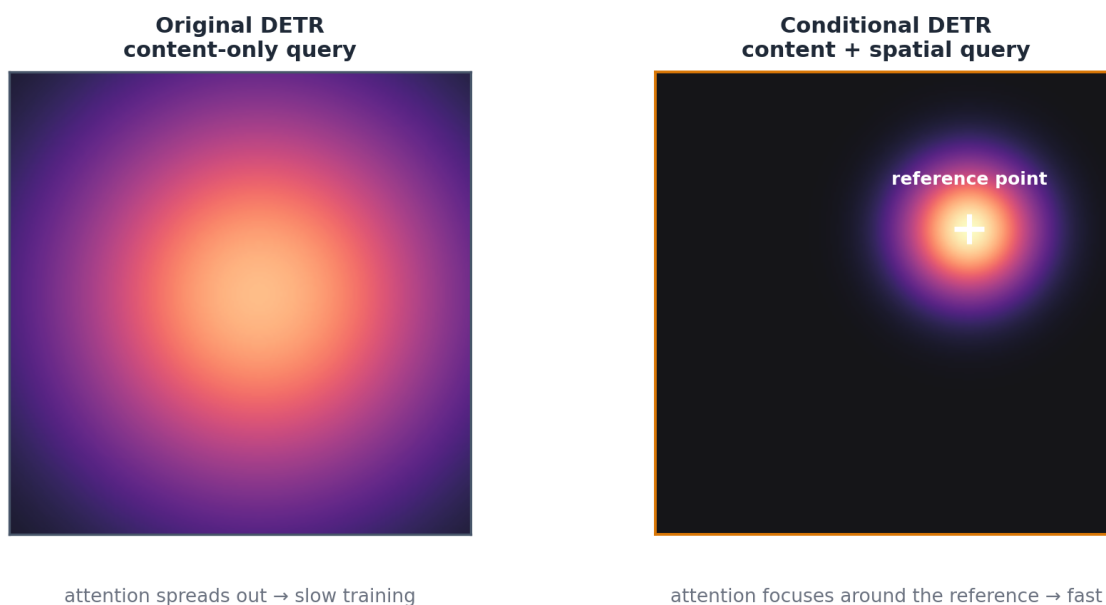


Figure 2 — Left: content-only query, diffuse attention (slow). Right: content + spatial query, attention focused around the reference point (fast).

3. The idea: separate the “what?” from the “where?”

The key insight of Conditional DETR (Meng et al., 2021) is to **take localization off the content embedding’s shoulders** and feed it to the query as a separate, explicit spatial signal. It builds this conditional spatial query in three steps.

1) Reference point. Each query is assigned a reference point, s — a 2D coordinate on the image, produced from the query positional embedding by a learned linear layer followed by a sigmoid.

2) Sine embedding. From the reference point it builds a sine embedding, p_s . Crucially, this is written in the same “language” (the same sine scheme) as the image’s positional encoding — we return to why in Section 4.

3) Conditional spatial query. From the decoder’s content output it learns a transform $T = FFN(f)$, and the spatial query is $q_s = T \odot p_s$ (element-wise product). T lets every query adapt the spatial search to its own content.

In cross-attention, both the query and the key then become the **concatenation of two parts**: the query from content (f) and the spatial term (q_s), the key from the memory content (k) and its positional encoding (p_k). This makes the attention weight split into two independent terms (Figure 3).

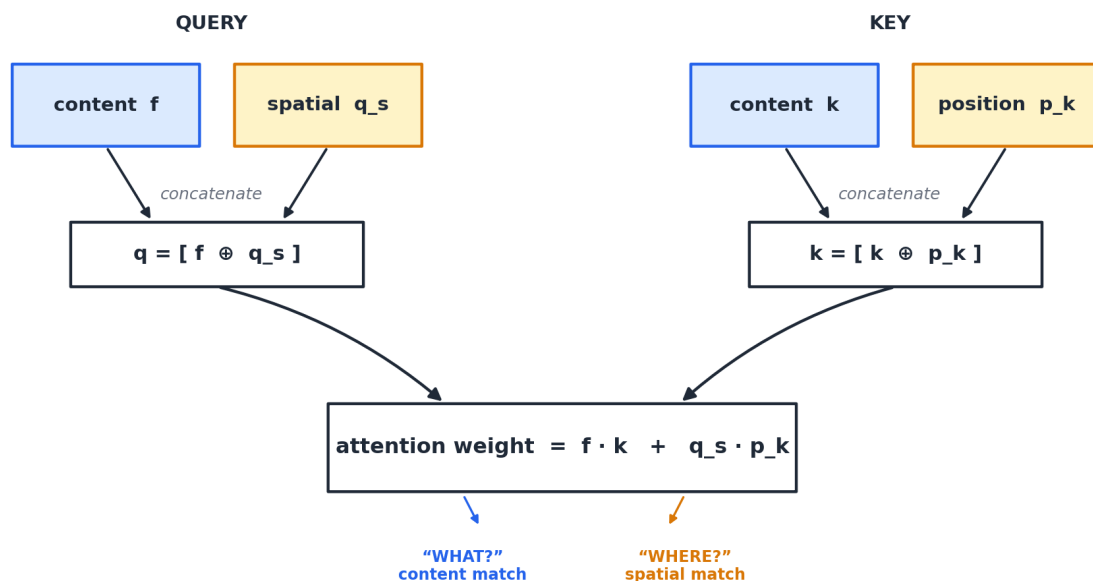


Figure 3 — The decoupled attention weight: content match and spatial match add up as separate terms.

Decoupling is the whole point

The first term answers “**WHAT?**” (appearance match), the second answers “**WHERE?**” (spatial-position match). The original DETR added these two effects into one query, so they got entangled; Conditional DETR **separates** them so each can act cleanly on its own.

4. Why does attention focus around the reference point?

The second term, $q_s \cdot p_k$, works because p_s (the reference-point embedding) and p_k (the grid-point embeddings) live in the **same sine space**. The dot product of two locations' sine embeddings is larger the closer the two locations are. So when we compare the reference-point embedding against the positional embedding of every location on the feature map, the product peaks precisely **around the reference point**.

Figure 4 shows this with a **real computation**: we plot the dot product between a reference point's embedding and the positional embeddings of a 30×30 grid. The attention concentrates cleanly around the reference point, and when the reference moves, the focus moves with it. That is why the query searches in the “right neighborhood” from the very first training steps instead of groping blindly — the source of the order-of-magnitude speed-up.

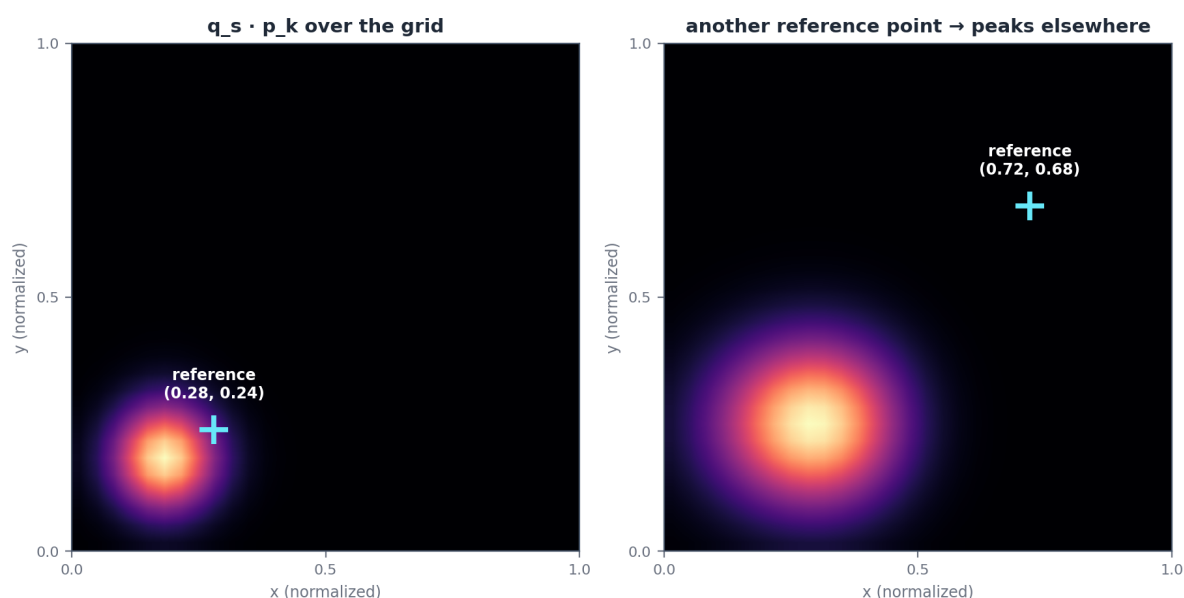


Figure 4 — Spatial attention computed for two different reference points. The bright blob always lands around the reference point (cyan +).

5. The box head: an offset from the reference point

The reference point also plays a role in box prediction. Instead of having the network guess the box’s absolute location from nothing, it only needs to learn the **offset** relative to the reference point:

$$b = \sigma(\text{FFN}(f) + [s_x, s_y, 0, 0])$$

The reference (x, y) coordinates are added to the predicted offset *before* the sigmoid (σ), and zero is added to the width/height terms. This “good starting point” further eases and speeds up learning: the model fine-tunes the box location rather than building it from scratch.

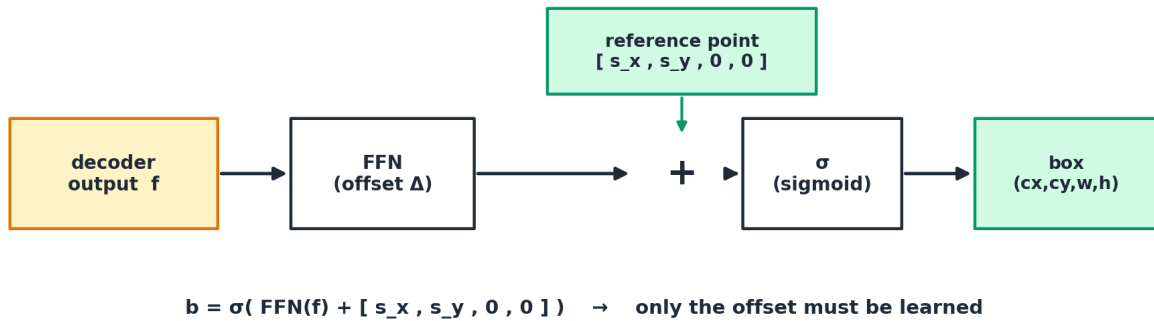


Figure 5 — The box head uses the reference point as a starting point and only learns the offset relative to it.

6. Summary

With a single, well-aimed change — **separating spatial search from content search** — Conditional DETR dramatically speeds up training while keeping DETR’s elegance: it remains an anchor-free, NMS-free, end-to-end detector. The authors report roughly **6.7×** faster convergence with ResNet-50/101 backbones, and up to **10×** faster with the stronger DC5 variants — a fraction of the original ~500 epochs suffices, at comparable accuracy.

Aspect	Original DETR	Conditional DETR
Cross-attention query	content only (+ shared query position)	content $\oplus$ conditional spatial (q_s)
Spatial localization	implicit, from the content embedding	explicit reference point + sine embedding
Attention weight	entangled	splits: $f \cdot k + q_s \cdot p_k$
Box prediction	absolute location	offset from the reference point
Convergence	~500 epochs	~6.7–10× faster

Connection to the code

In the attached `detr.py`, `ConditionalDecoderLayer` and `ConditionalCrossAttention` implement exactly this mechanism: the shared reference points, the $T \odot p_s$ spatial query, and the concatenated, decoupled cross-attention.